Repaso del día 4

16 preguntas sobre trilema de blockchain, MEV, PBS, encrypted mempools, gas y nonce

Las 16 preguntas — Repaso día 4

1. ¿Qué tres propiedades plantea el trilema de blockchain y por qué son difíciles de tener juntas?
2. ¿Por qué una red muy descentralizada y muy segura sacrifica escalabilidad?
3. ¿Por qué una red muy descentralizada y muy escalable sacrifica seguridad?
4. ¿Por qué una red muy segura y muy escalable sacrifica descentralización?
5. ¿Qué es el MEV y qué tipos de ataques permite?
6. ¿Qué propone Proposer-Builder Separation (PBS) y qué problema ataca?
7. ¿Quiénes son los actores principales en el ecosistema PBS y qué gana cada uno?
8. ¿Por qué se dice que en PBS el proposer firma a ciegas y qué le impide hacer front-running?
9. ¿PBS elimina realmente el front-running para el usuario?
10. ¿Qué proponen los encrypted mempools y por qué evitan el front-running?
11. ¿Qué propone LUCID (EIP-8184) y por qué es la apuesta de futuro?
12. ¿Cómo intentaría un atacante hacer front-running con PBS, y por qué no funciona en un encrypted mempool?
13. ¿Qué es el cifrado César, es criptografía simétrica o asimétrica?
14. ¿Qué diferencia hay entre 'Tarifa base mínima' y 'Tarifa de prioridad' en MetaMask?
15. ¿Qué es el nonce de Ethereum y qué propósito cumple?
16. Si una transacción se queda atascada por gas demasiado bajo, ¿cómo se desbloquea?

PREGUNTA 1 / 16

El trilema de blockchain

Pregunta

¿Qué tres propiedades plantea el trilema de blockchain y por qué son difíciles de tener juntas?

Respuesta razonada

Seguridad (resistencia a ataques y fallos), escalabilidad (procesar muchas transacciones rápido y barato) y descentralización (que cualquiera pueda ser nodo sin barreras). Lo planteó Vitalik Buterin: en la práctica, optimizar dos siempre obliga a sacrificar la tercera. Es una observación empírica sobre las elecciones de diseño, no un teorema matemático estricto.

PREGUNTA 2 / 16

Descentralización + seguridad

Pregunta

¿Por qué una red muy descentralizada y muy segura sacrifica escalabilidad?

Respuesta razonada

Porque cada nodo completo valida todas las transacciones, lo que asegura máxima seguridad pero limita la velocidad al nodo más lento. Para que cualquiera pueda participar (incluso con conexión débil), los bloques deben ser pequeños y espaciados. Resultado: pocas transacciones por segundo. Ejemplo claro: Bitcoin (~7 TPS, bloques cada 10 minutos), que ha necesitado sidechains y Layer 2 para agilizar.

PREGUNTA 3 / 16

Descentralización + escalabilidad

Pregunta

¿Por qué una red muy descentralizada y muy escalable sacrifica seguridad?

Respuesta razonada

Si cualquiera puede levantar un nodo con pocos recursos y la red procesa miles de TPS, cada nodo debe validar muy rápido y con poco coste. Eso obliga a relajar requisitos de validación, abriendo la puerta a transacciones inválidas o forks mal gestionados. Ejemplo: Solana en 2021 buscaba alta escalabilidad y muchos nodos, y sufrió una caída de 17 horas por saturación al no poder eliminar un fork.

PREGUNTA 4 / 16

Seguridad + escalabilidad

Pregunta

¿Por qué una red muy segura y muy escalable sacrifica descentralización?

Respuesta razonada

Procesar miles de TPS con seguridad fuerte exige nodos con gran capacidad de cómputo y ancho de banda. Solo unos pocos actores con infraestructura potente pueden ser validadores; el coste de entrada excluye a los nodos pequeños. Resultado: red más centralizada en pocas manos, con riesgo de censura y colusión. Ejemplo: redes permisionadas empresariales (Hyperledger Fabric, R3 Corda), rápidas y seguras pero con descentralización limitada por diseño.

PREGUNTA 5 / 16

MEV — Maximal Extractable Value

Pregunta

¿Qué es el MEV y qué tipos de ataques permite?

Respuesta razonada

MEV es el beneficio que un minero o validador puede extraer reordenando, incluyendo o excluyendo transacciones del bloque que produce. No es un fallo: es consecuencia de que UN solo actor decida qué entra y en qué orden. Tipos clásicos: front-running (meter una tx propia antes de una grande para aprovechar el cambio de precio), back-running (ejecutar justo después de algo que mueve el mercado) y sandwich attack (comprar antes y vender justo después de una tx grande, atrapando al usuario en medio).

PREGUNTA 6 / 16

PBS — la idea

Pregunta

¿Qué propone Proposer-Builder Separation (PBS) y qué problema ataca?

Respuesta razonada

Hasta PBS, un único actor (validador o minero) decidía qué transacciones incluir y en qué orden — tenía el monopolio del MEV. PBS rompe ese monopolio dividiendo el rol en dos: el builder construye el bloque más rentable (incluyendo MEV legítimo) y el proposer (validador) solo recibe bloques ya armados, elige el que más le paga y lo firma. Si el validador no ve las transacciones, no puede aprovecharse de ellas.

PREGUNTA 7 / 16

Los actores de PBS

Pregunta

¿Quiénes son los cuatro actores principales en el ecosistema PBS y qué gana cada uno?

Respuesta razonada

Proposer: validador con 32 ETH en stake elegido para el slot, gana la recompensa del bloque, los tips y el pago del builder ganador. Builder: especialista en armar bloques rentables con MEV, gana el MEV extraído menos lo que paga al proposer. Relay: intermediario neutral que custodia los bloques y muestra al proposer solo cabecera y precio, gana en reputación y a veces tarifas. Searcher: detecta oportunidades de MEV concretas y envía bundles a los builders, gana el margen del MEV menos lo que paga al builder.

PREGUNTA 8 / 16

PBS — firma a ciegas

Pregunta

¿Por qué se dice que en PBS el proposer firma a ciegas y qué le impide hacer front-running?

Respuesta razonada

El relay solo le entrega al proposer la cabecera del bloque y el precio que paga el builder, no el contenido. El proposer firma sin saber qué transacciones contiene. Si firmara dos bloques distintos para el mismo slot intentando hacer trampa, perdería su stake (slashing — ~32 ETH). Además, el bloque incluye el pago del builder de forma atómica: si firma cobra, si no firma no cobra. Por eso el proposer no puede hacer front-running por sí mismo.

PREGUNTA 9 / 16

Lo que PBS NO resuelve

Pregunta

¿PBS elimina realmente el front-running para el usuario?

Respuesta razonada

No. PBS rompe el monopolio del validador, pero el builder sigue pudiendo hacer MEV — es literalmente su modelo de negocio. Un builder ve la transacción de un usuario en el mempool, monta un sandwich y paga al proposer un fee desproporcionado para que su bloque gane la subasta. La competencia entre builders fuerza a transferir casi todo el MEV al proposer, pero el usuario sigue siendo víctima. PBS democratiza el MEV; no lo elimina.

PREGUNTA 10 / 16

Encrypted Mempools — la idea

Pregunta

¿Qué proponen los encrypted mempools y por qué evitan el front-running?

Respuesta razonada

Las transacciones viajan cifradas por el mempool. Nadie — ni siquiera el builder o el validador — puede leer su contenido hasta que están incluidas en un bloque. Si el contenido es opaco, no se puede hacer front-running ni sandwich porque no se sabe qué intenta hacer cada usuario. La transacción se cifra de modo que pueda incluirse en un bloque ANTES de descifrarla; solo después de ordenado y firmado el bloque, se descifra y se ejecuta.

PREGUNTA 11 / 16

EIP-8184 LUCID

Pregunta

¿Qué propone LUCID (EIP-8184) y por qué es la apuesta de futuro?

Respuesta razonada

LUCID lleva el encrypted mempool al núcleo del protocolo de Ethereum. Las transacciones viajan como sealed transactions (ST) — públicas pero opacas — y se descifran solo después de fijar las decisiones de ordenamiento. Mecanismos clave: commit-before-reveal (los builders se comprometen antes de ver el contenido), key publishers (liberan la clave tras el scheduling), ST tickets, bundles y ToB fees. Es encryption-agnostic y, al estar en el protocolo, no depende de relays externos como MEV-Boost.

PREGUNTA 12 / 16

Trampas con PBS y con encrypted mempool

Pregunta

¿Cómo intentaría un atacante hacer front-running con PBS, y por qué no funciona en un encrypted mempool?

Respuesta razonada

Con PBS: el atacante actúa como builder, mete su transacción de front-running en el bloque y paga al proposer un fee desproporcionado para ganar la subasta. Funciona — PBS no protege al usuario. Con encrypted mempool: el front-running clásico no funciona porque el contenido está cifrado y nadie ve qué intenta hacer cada tx. Quedan vías indirectas (comprometer al key publisher, censurar transacciones, análisis de metadatos) pero todas son mucho más difíciles que un front-running ordinario.

PREGUNTA 13 / 16

Cifrado César y ROT13

Pregunta

¿Qué es el cifrado César y qué es concretamente ROT13?

Respuesta razonada

El cifrado César es una técnica de cifrado simétrico que usaba Julio César para comunicarse con sus generales: desplaza cada letra del alfabeto un número fijo de posiciones. Si el desplazamiento es 13 se llama ROT13, una variante curiosa porque al aplicarla dos veces se vuelve al texto original (26/2 = 13). Es trivial de romper hoy — sirve solo como ejemplo histórico de criptografía simétrica, donde la misma clave (el desplazamiento) cifra y descifra.

PREGUNTA 14 / 16

Tarifa base y tarifa de prioridad

Pregunta

¿Qué diferencia hay entre 'Tarifa base mínima' y 'Tarifa de prioridad' en MetaMask?

Respuesta razonada

La tarifa base mínima (max base fee) es el máximo en Gwei por unidad de gas que estás dispuesto a pagar para cubrir la base fee dinámica de la red. Ethereum la ajusta en cada bloque; si la base real es menor, te devuelven la diferencia. La tarifa de prioridad (priority fee) es la propina al validador para que priorice tu transacción frente a otras del mempool. Si pones un fee total muy bajo, los validadores la ignoran si otras pagan más, y tu transacción queda 'pendiente' indefinidamente.

PREGUNTA 15 / 16

El nonce de Ethereum

Pregunta

¿Qué es el nonce de Ethereum y qué propósito cumple?

Respuesta razonada

Es un número secuencial que identifica cada transacción enviada por una cuenta. Empieza en 0 y se incrementa en +1 con cada nueva transacción. Garantiza el orden exacto de las transacciones de esa cuenta y evita repeticiones o duplicados (ataques de replay). Las transacciones deben minarse en orden: si se salta un número, las siguientes quedan pendientes hasta que la anterior se confirme. No confundir con el nonce del PoW, que es algo distinto.

PREGUNTA 16 / 16

Replace-by-fee

Pregunta

Si una transacción se queda atascada por gas demasiado bajo, ¿cómo se desbloquea?

Respuesta razonada

Con la técnica replace-by-fee. Pasos: 1) identificar el nonce bloqueado (en Etherscan o en MetaMask → Configuración avanzada). 2) Crear una nueva transacción con el mismo nonce, al mismo destino o a tu propia dirección, con un gas más alto. La nueva sobrescribe la antigua. Variante 'cancelar': enviar una tx vacía (0 ETH a ti mismo) con el mismo nonce y gas alto; si se mina, libera el nonce y desbloquea las siguientes. Resultado: la cola vuelve a fluir.

Etherscan

El explorador de Ethereum: cómo leer la blockchain pública

INTRO

¿Qué es Etherscan?

Definición

Etherscan es un explorador de bloques (block explorer) para Ethereum. Es una página web que se conecta a un nodo de la red y muestra de forma legible toda la información pública: transacciones, bloques, direcciones, smart contracts, tokens, NFTs…

La analogía

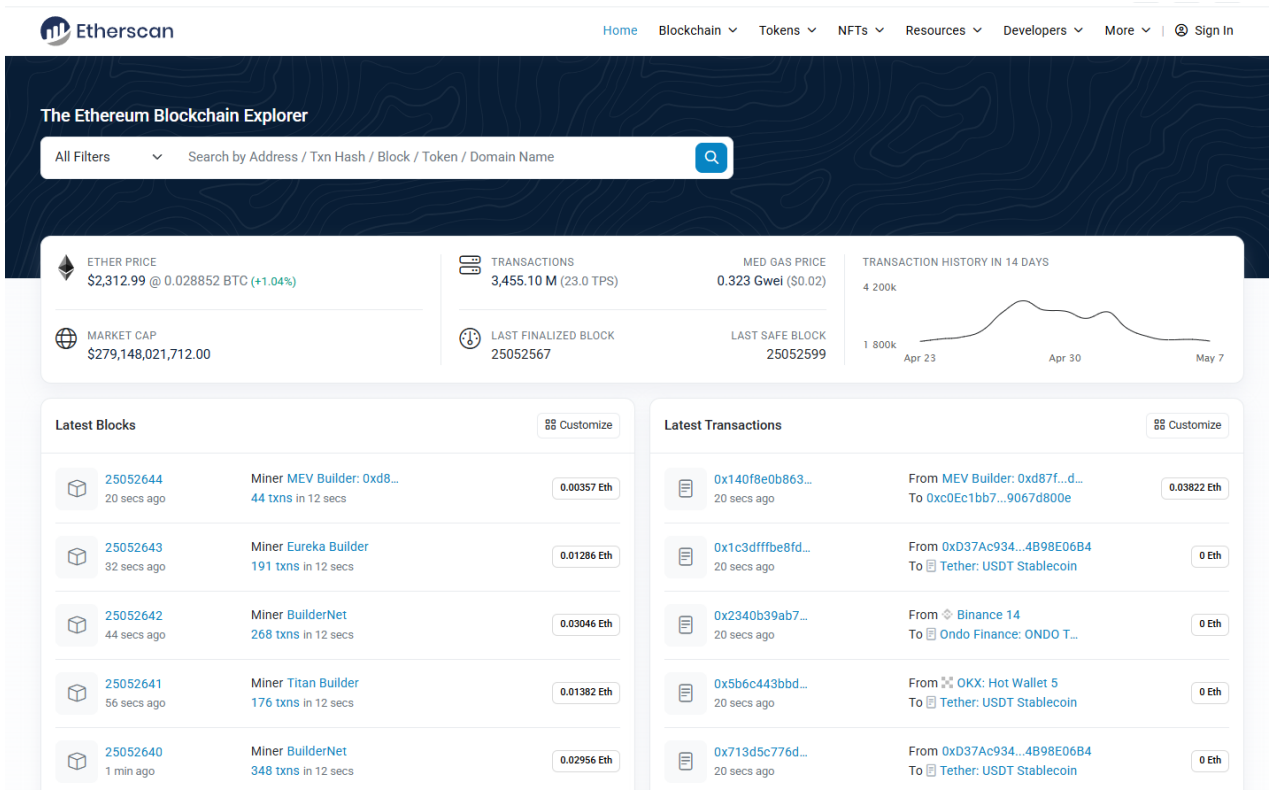
Imagina una ventana al libro mayor de Ethereum. Etherscan no es Ethereum, es un visor: lee lo que ya está en la blockchain (que es pública) y lo presenta con tablas, gráficos y enlaces. Cualquier dato que veas en Etherscan está también en cualquier otro nodo o explorador de la red.

URLs principales

- **Mainnet:** https://etherscan.io
- **Testnet Sepolia:**
https://sepolia.etherscan.io (la usaremos en clase)

La página principal

Qué se ve nada más entrar



Elementos clave

- **Barra de búsqueda:** acepta tx hash, dirección, número de bloque, ENS o nombre de token.

- **Precio actual de ETH y gas tracker:**
cuánto cuesta la red ahora mismo en Gwei.

- **Últimos bloques:**
se ven los últimos producidos en tiempo real.

- **Últimas transacciones:**
transacciones recientes incluidas en bloques.

- **Menú superior:**
Blockchain, Tokens, NFTs, Resources, Developers.

USO BÁSICO

Cómo buscar en Etherscan

La barra de búsqueda acepta cuatro cosas

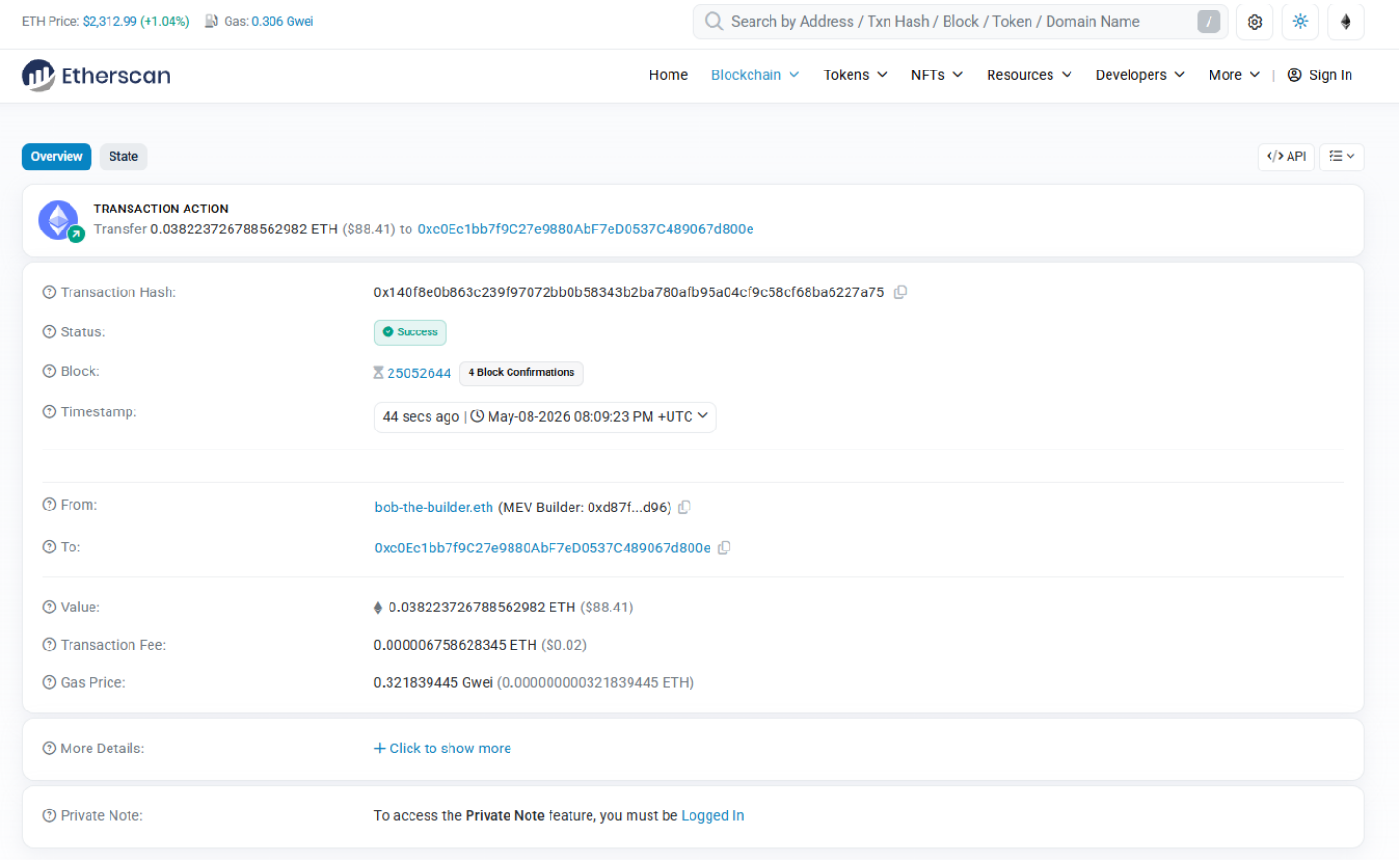
Qué buscas	Aspecto / formato	Ejemplo
Transacción (tx hash)	Cadena de 66 caracteres que empieza por 0x	`0x4a1c...`
Dirección (address)	Cadena de 42 caracteres que empieza por 0x	`0xd35d...`
Bloque	Número entero (el bloque N de la cadena)	`21000000`
ENS o token	Nombre legible (ENS) o ticker de token	`vitalik.eth, USDC`

Truco práctico

Etherscan reconoce el formato automáticamente: pega lo que sea y te lleva a la página correspondiente. Si te equivocas en un solo carácter, no encontrará nada — todos los identificadores son hashes y un bit cambiado los invalida.

Anatomía: visión general

Lo que verás al abrir una transacción



Cada transacción tiene su propia URL permanente. Una vez incluida en un bloque, su información es inmutable y siempre estará accesible.

TRANSACCIÓN

Campos clave

Status: Success / Failed. Una transacción puede minarse y aun así fallar (revert) — aún paga el gas.

Block: número del bloque que la incluye + número de confirmaciones acumuladas desde entonces.

Timestamp: fecha y hora exactas en que se incluyó.

From / To: dirección que firma (origen) y dirección destino. Si To es un contrato, se ve el método invocado.

Value: ETH transferido. Puede ser 0 si la transacción solo invoca un contrato sin enviar ether.

Transaction Fee: ETH total pagado al validador (gas usado × precio del gas). En USD también.

Gas Used / Gas Price / Gas Limit: cuánto gas consumió, a qué precio (Gwei), y el máximo permitido por el remitente.

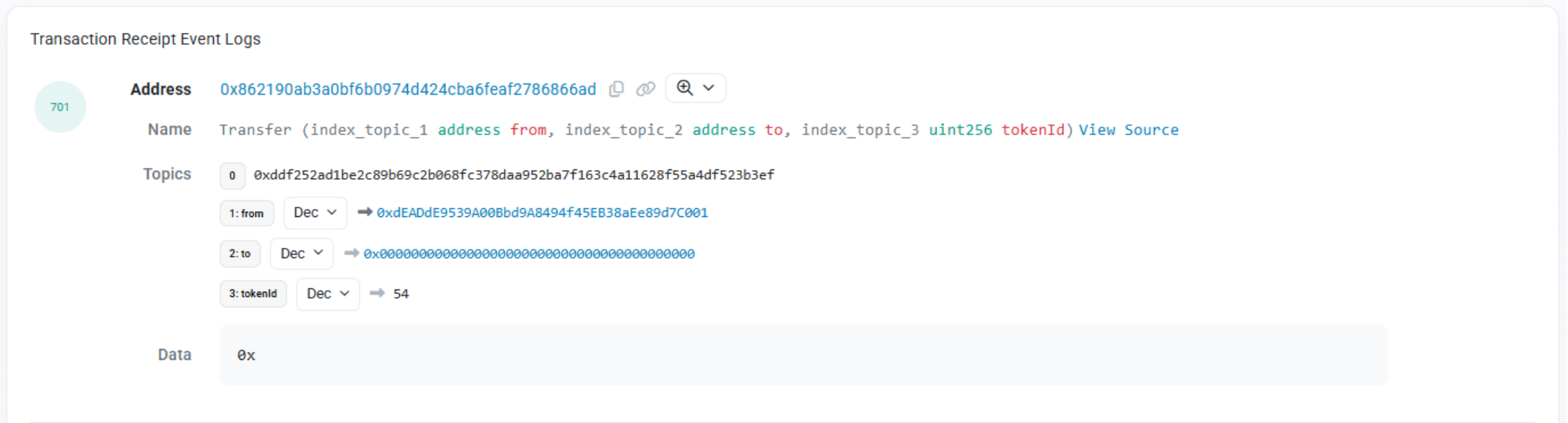
Nonce: número secuencial de la transacción dentro de la cuenta origen (recordad el día 3).

Input Data: datos enviados al contrato — Etherscan los decodifica si conoce el ABI.

Logs / Events — qué ha cambiado

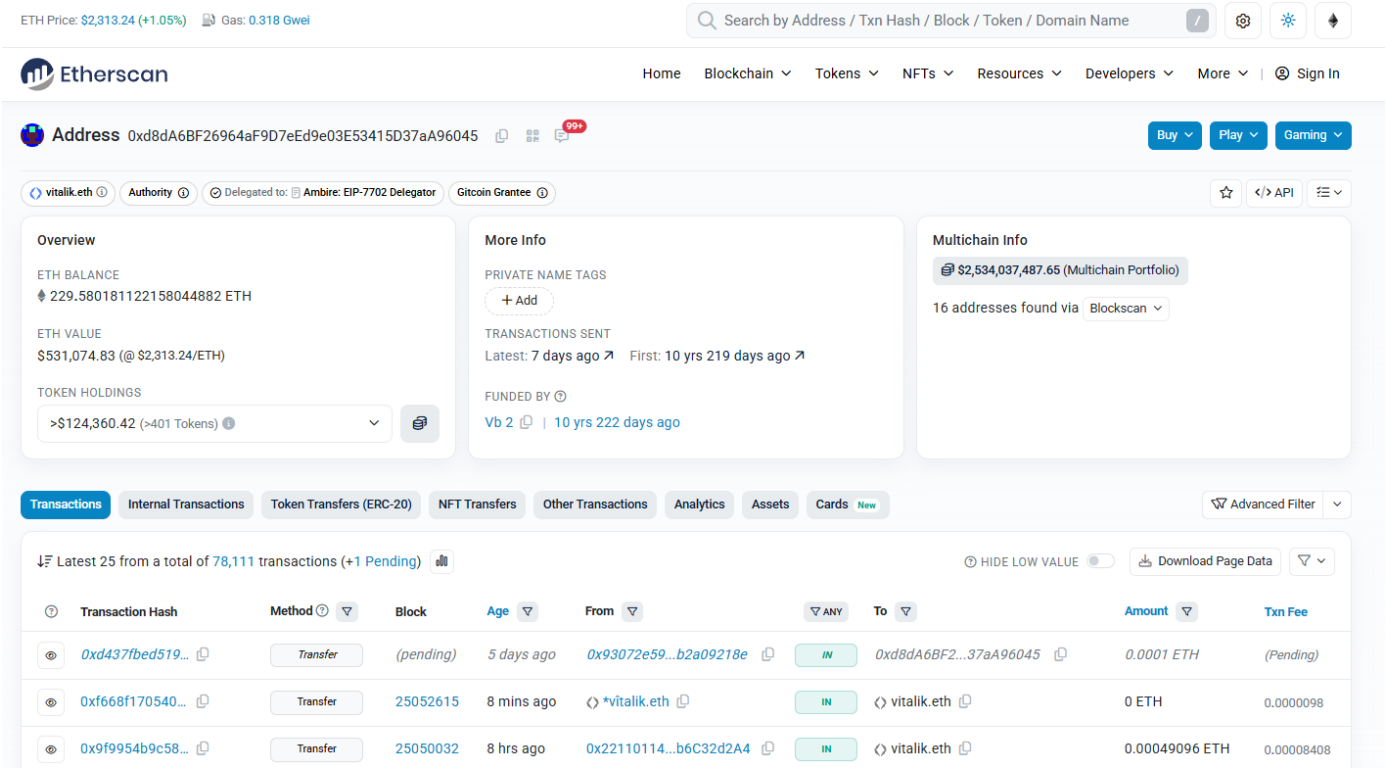
La pestaña Logs

Cuando una transacción interactúa con un smart contract, el contrato puede emitir eventos para señalar qué ha ocurrido (transferencia de un token, voto en una DAO, asset creado…). Etherscan los muestra en la pestaña 'Logs' con todos los parámetros decodificados.



Cómo se ve un wallet en Etherscan

La página de una dirección



Información que se ve

- Address (0x…)
- ENS si lo tiene (vitalik.eth)
- Balance de ETH actual
- Equivalente en USD
- Total de transacciones
- Tokens que tiene (ERC-20)
- NFTs que tiene
- Si es un contrato o un EOA

EOA vs Contrato: *una wallet normal es una EOA (cuenta externa, controlada por una clave privada). Una dirección de contrato es código desplegado. Etherscan los distingue visualmente.*

Las pestañas de una dirección

Transactions: histórico completo de transacciones donde esta dirección es origen o destino.

Internal Transactions: transferencias de ETH causadas indirectamente por la ejecución de un contrato (no son tx propias, sino efectos).

ERC-20 Token Txns: movimientos de tokens fungibles (USDC, DAI, USDT…).

ERC-721 / ERC-1155 Token Txns: movimientos de NFTs.

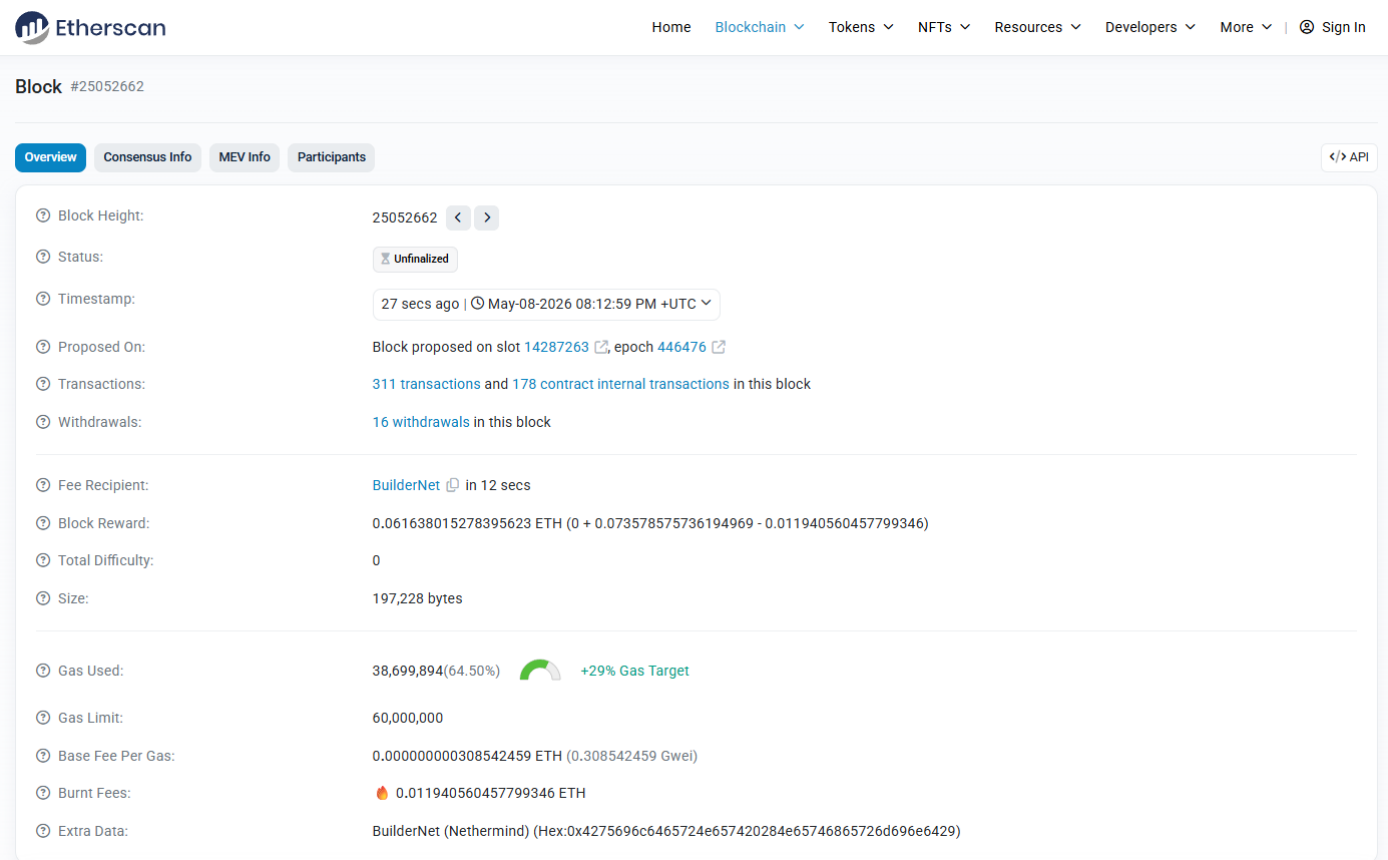
Analytics: gráficos de actividad (transacciones por día, gas pagado, balance histórico).

Comments: comentarios públicos que otros usuarios pueden dejar (útil para identificar wallets de exchanges, scammers, etc.).

BLOQUE

Vista de un bloque

Lo que verás al abrir un bloque



Campos del bloque

- **Block Height:** número de bloque
- **Status:**
Finalized / Safe
- **Timestamp:**
cuándo se creó
- **Proposed By:**
qué validador lo propuso
- **Transactions:**
cuántas y enlace al detalle
- **Gas Used / Limit**

- **Base Fee per Gas**

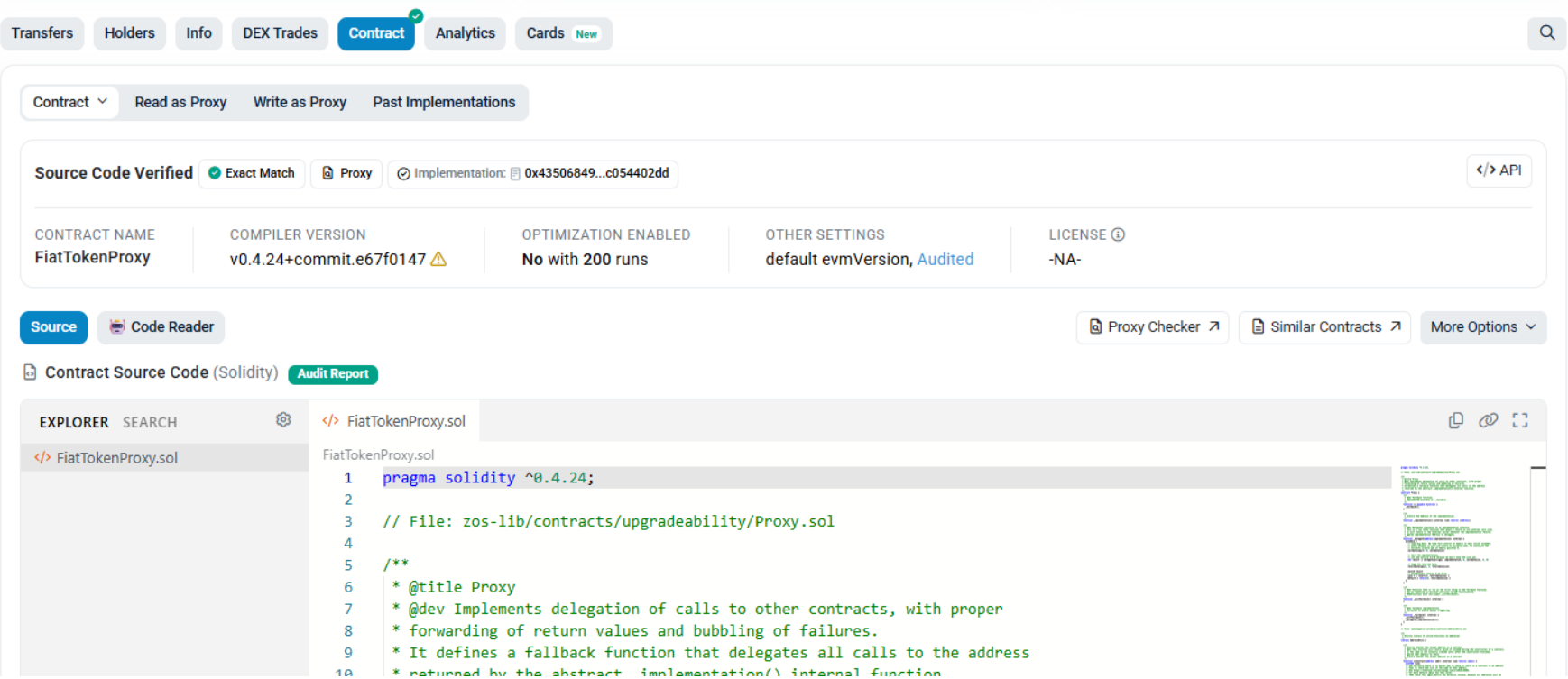
- **Block Reward / Burnt Fees**

- **Hash y Parent Hash**

Ver el código verificado

La pestaña Contract

Si la dirección que estás viendo es un contrato y su autor publicó el código fuente, Etherscan lo muestra y permite verificar que el bytecode desplegado coincide con ese código. Esto es lo que llamamos un contrato 'verified'.

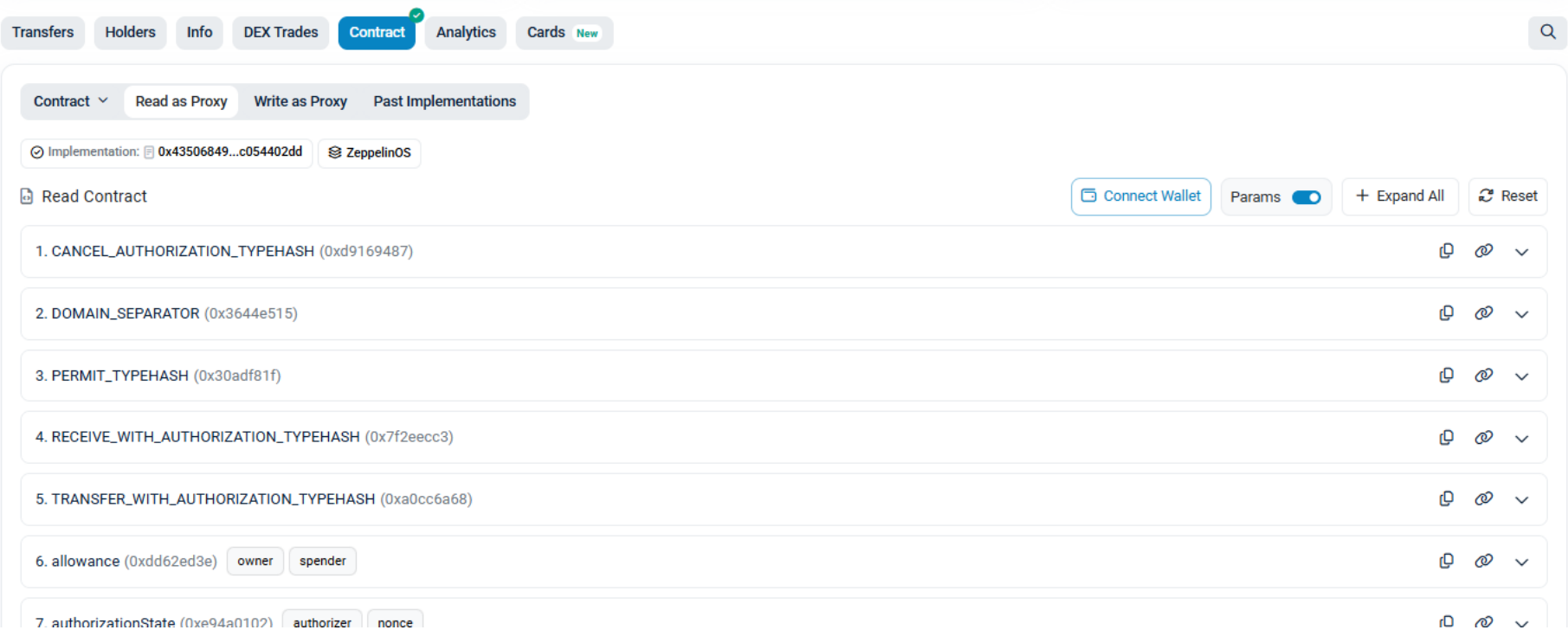


¿Por qué importa? Si el contrato no está verificado, solo verás bytecode — nadie puede auditarlo fácilmente. Verificarlo es un acto de transparencia del autor.

Read Contract — leer datos

Interactuar sin gastar gas

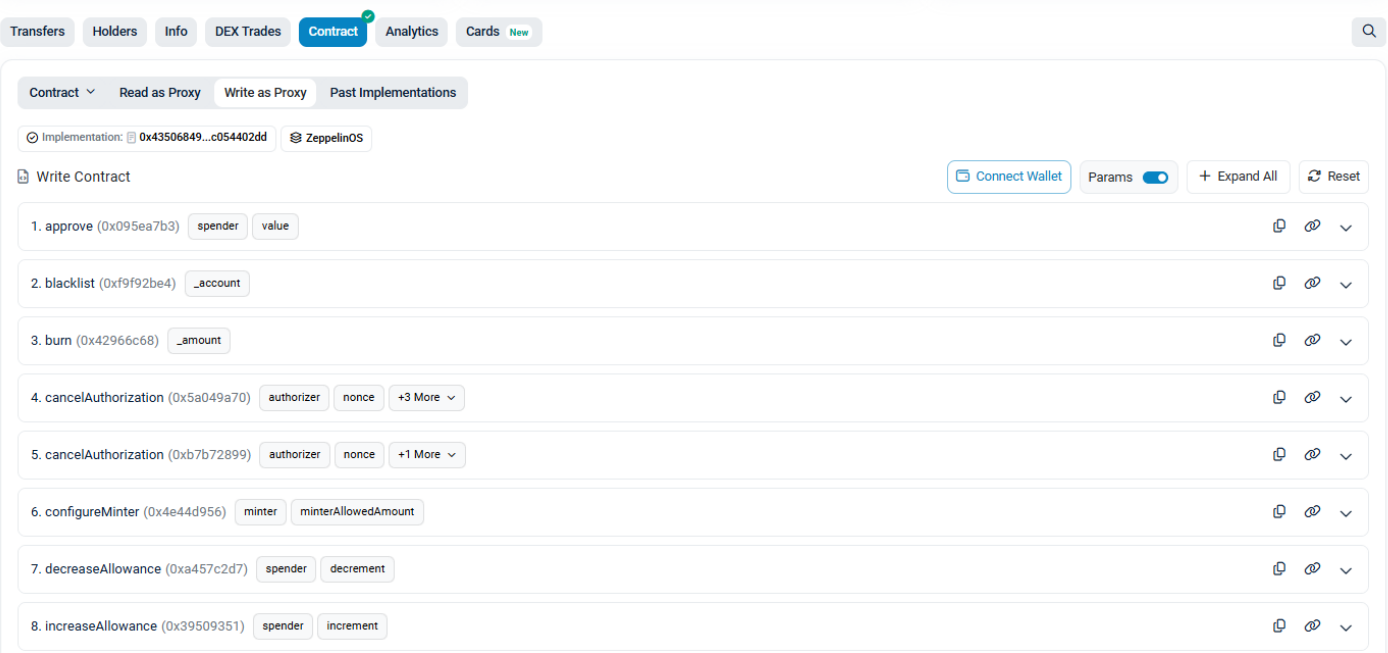
En la pestaña 'Read Contract' Etherscan lista todas las funciones de solo lectura (view/pure) del contrato. Puedes invocarlas directamente desde el navegador sin firmar ni pagar gas — solo se consulta el estado.



Write Contract — modificar estado

Interactuar gastando gas

La pestaña 'Write Contract' lista las funciones que modifican el estado. Para usarlas hay que conectar MetaMask: Etherscan firma la transacción con tu wallet y la envía a la red. Pagas gas como en cualquier otra transacción.



Aviso para los alumnos: *en mainnet, cualquier interacción cuesta dinero real. En clase usaremos siempre Sepolia (testnet) donde el ETH es de prueba.*

Tokens y NFTs en Etherscan

Cómo navegar tokens

Cada token (ERC-20, ERC-721, ERC-1155) **es en realidad un smart contract.** Etherscan le añade una vista especial.

Lo que se ve en la página de un token:

- Nombre, símbolo y total supply
- Precio actual (si está listado en exchanges)
- Holders: cuántas direcciones tienen este token y cuánto
- Transfers: histórico de transferencias
- Información del contrato (la pestaña Contract con Read/Write)

Para NFTs:

se puede ver cada token individual (cada NFT tiene un ID único), su propietario actual, el histórico de ventas, los metadatos y la imagen asociada.

ECOSISTEMA

Etherscan en otras redes y exploradores similares

Mismo equipo, distintas redes

Red	Explorador	URL
Ethereum mainnet	Etherscan	`etherscan.io`
Sepolia (testnet)	Etherscan Sepolia	`sepolia.etherscan.io`
BNB Chain	BscScan	`bscscan.com`
Polygon	PolygonScan	`polygonscan.com`
Arbitrum	Arbiscan	`arbiscan.io`
Optimism	Optimistic Etherscan	`optimistic.etherscan.io`

Otros exploradores destacables

Blockscan (busca en todas las redes a la vez), Blockchair (multicadena), DeBank (visión por wallet), Tenderly (debugging para devs).

ACTIVIDAD

Práctica guiada en Sepolia Etherscan

Por parejas, 30 minutos

1. Buscar tu propia dirección de MetaMask en sepolia.etherscan.io. ¿Cuántas transacciones tienes? ¿Qué tokens?

2. Abrir una de tus transacciones de envío de ETH al compañero. Identificar: From, To, Value, Gas Used, Gas Price, Transaction Fee, Nonce.

3. Buscar el contrato del USDC de Sepolia. ¿Está verificado? ¿En qué versión de Solidity está escrito?

4. En la pestaña Read Contract del USDC: invocar `balanceOf` con tu propia dirección. ¿Cuántos USDC tienes?

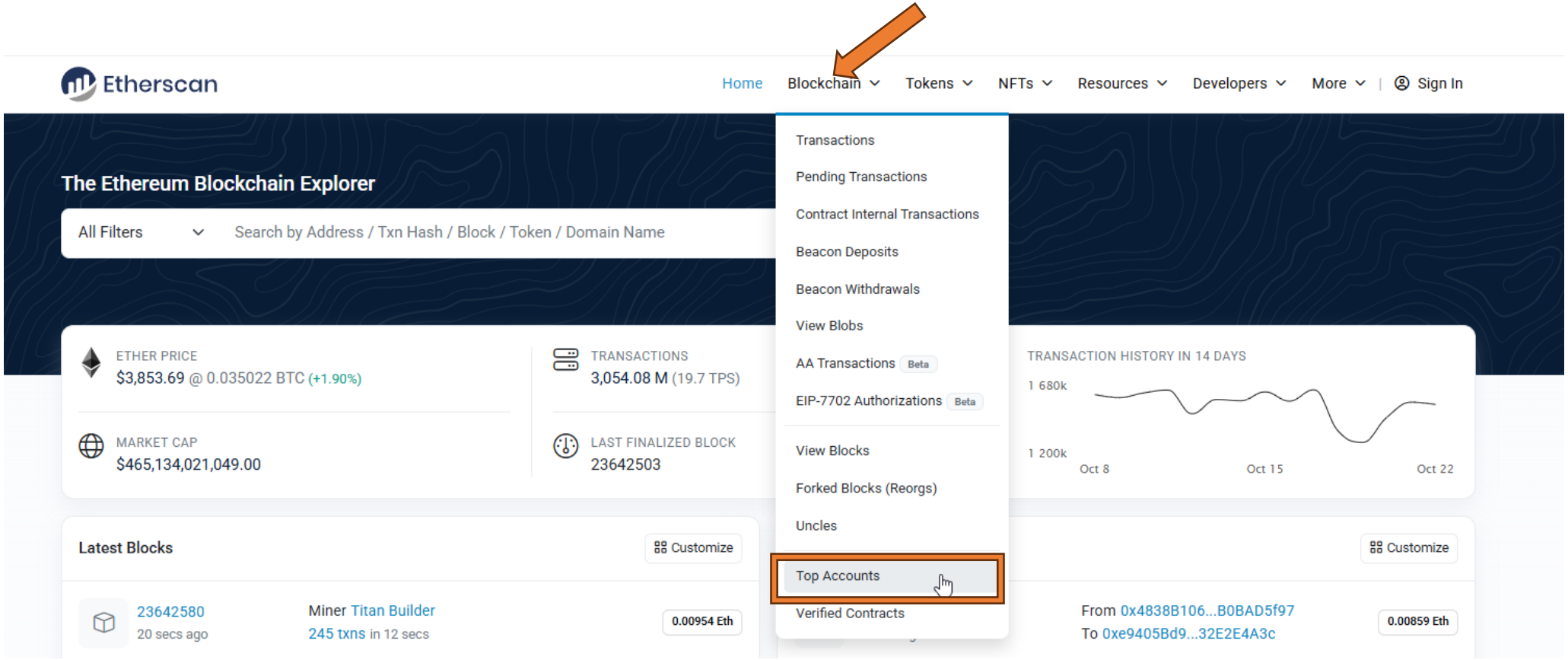
5. Ir al último bloque producido. ¿Cuántas transacciones lleva? ¿Cuál es el hash del bloque anterior?

6. **Bonus:** buscar la dirección `vitalik.eth` en mainnet etherscan.io. ¿Cuántos ETH tiene? ¿Cuántos NFTs?

cotilleos

Etherscan
ETH Balance ↓
1 0x8be...7f81
2 0x028...ebd4
3 0x5e5...3b1f
4 0x73c...a2b9
5 0x9fd...c6e2
6 0x61a...d7f3
7 0x3b2...1e8a
8 0x7c1...9b2d
9 0x6e4...f3a7
10 0x1a2...c4d5

Cotilleos


Etherscan
Home
Blockchain
Tokens
NFTs
Resources
Developers
More
Sign In
The Ethereum Blockchain Explorer
All Filters
Search by Address / Txn Hash / Block / Token / Domain Name
Transactions
Pending Transactions
Contract Internal Transactions
Beacon Deposits
Beacon Withdrawals
View Blobs
AA Transactions Beta
EIP-7702 Authorizations Beta
View Blocks
Forked Blocks (Reorgs)
Uncles
Top Accounts
Verified Contracts
ETHER PRICE
$3,853.69 @ 0.035022 BTC (+1.90%)
TRANSACTIONS
3,054.08 M (19.7 TPS)
TRANSACTION HISTORY IN 14 DAYS
1 680k
1 200k
Oct 8
Oct 15
Oct 22
MARKET CAP
$465,134,021,049.00
LAST FINALIZED BLOCK
23642503
Latest Blocks
Customize
Customize
23642580
20 secs ago
Miner Titan Builder
245 txns in 12 secs
0.00954 Eth
From 0x4838B106...B0BAD5f97
To 0xe9405Bd9...32E2E4A3c
0.00859 Eth

Cotilleos

Etherscan

Home | Blockchain | Tokens | NFTs | Resources | Developers | More | Sign In

Top Accounts by ETH Balance

API

Sponsored: Coinpoker: Instant Withdrawal Crypto Gaming & Poker - $2,000 Bonus! Play Now!

More than 1,999,999 accounts found (120,698,331.593 ETH)
(Showing the last 10,000 top accounts only)

Download Page Data | First | Page 1 of 400 | Last

#	Address	Name Tag	Balance	Percentage	Txn Count
1	0x00000000...03d7705Fa	Beacon Deposit Contract	71,557,814.60089408 ETH	59.28649854%	712,346
2	0xC02aaA39...83C756Cc2	Wrapped Ether	2,167,164.32252315 ETH	1.79552136%	20,975,662
3	0xBE0eB53F...2404d33E8	Binance 7	1,996,008.37056349 ETH	1.65371662%	2,404
4	0x40B38765...6e418E489	Robinhood	1,177,794.80417432 ETH	0.97581697%	343
5	0x0E58e899...893d9BfCD	Upbit 41	814,827.65303938 ETH	0.67509438%	36,396
6	0x8315177a...4DBd7ed3a	Arbitrum: Bridge	805,953.59044153 ETH	0.66774211%	191
7	0x49048044...fAF74E97e	Base: Base Portal	742,272.57950738 ETH	0.61498164%	1,115,838
8	0xF977814e...97441aceC	Binance: Hot Wallet 20	638,622.37543846 ETH	0.52910622%	19,090
9	0x47ac0Fb4...507a6D503	Binance: Binance-Peg Tokens	554,999.05638856 ETH	0.45982330%	475
10	0xE92d1A43...88f85226f	Bitfinex 19	450,118.33469272 ETH	0.37292838%	554
11	0x8D05d992...78EaE6FcF	Kraken: Cold Wallet 3	329,444.01838939 ETH	0.27294828%	78
12	0xcA8Fa8f0...97c8aFfCa		325,000.48318378 ETH	0.26926676%	268
13	0x3BfC20f0...F16540258	Polkadot: MultiSig	306,276.63553761 ETH	0.25375383%	579
14	0xAFCD96e5...ACD45C5dA	Gemini: Cold Wallet	293,763.80218899 ETH	0.24338680%	872

Custodial Wallets

RECORDATORIO

Custodial Wallets

¿Qué son?

Wallets donde el usuario NO controla las claves privadas. La custodia la lleva un tercero (un exchange tipo Coinbase o Binance, o un proveedor especializado).

✅ VENTAJAS

- Recuperación de acceso si se pierde la contraseña
- Experiencia de usuario más sencilla
- Integración fiat ↔ cripto integrada (compra con tarjeta)
- Sin riesgo de perder las 12 palabras

❌ DESVENTAJAS

- El usuario NO tiene soberanía total sobre sus fondos
- Riesgo de hackeo del custodio (Mt. Gox, FTX...)
- Posible bloqueo de cuentas o censura
- Dependencia de la honestidad de un tercero
- "Not your keys, not your coins"

Debate – Custodial Wallet

Comparar este tipo de wallets con Metamask

Recordad los términos:

- Seguridad
- Sencillez
- Accesibilidad
- Utilidad
- Protección ante errores o pérdida de la clave

Smart Contracts

El computador mundial: cómo Ethereum convirtió blockchain en una máquina programable

ORIGEN

El nacimiento de los Smart Contracts

2013 — Vitalik Buterin (19 años) publica el whitepaper de Ethereum. Propone evolucionar Bitcoin a algo más flexible: una blockchain capaz de ejecutar pequeños programas que llama ***contratos inteligentes.***

2014 — La comunidad Bitcoin lo ignora. Vitalik recauda fondos vía ICO junto a Joseph Lubin, Gavin Wood, Charles Hoskinson y otros. Recaudan 18.4M USD en BTC.

2015 — Lanzamiento de Ethereum (mainnet "Frontier" en julio). Desde entonces es el referente de smart contracts y blockchain programable.

2026 — Más del 60% de todo el valor en blockchain corre sobre Ethereum o redes EVM-compatibles (Polygon, BSC, Arbitrum, Optimism, Base...).

LA IDEA

Blockchain como máquina de estados

La visión de Vitalik

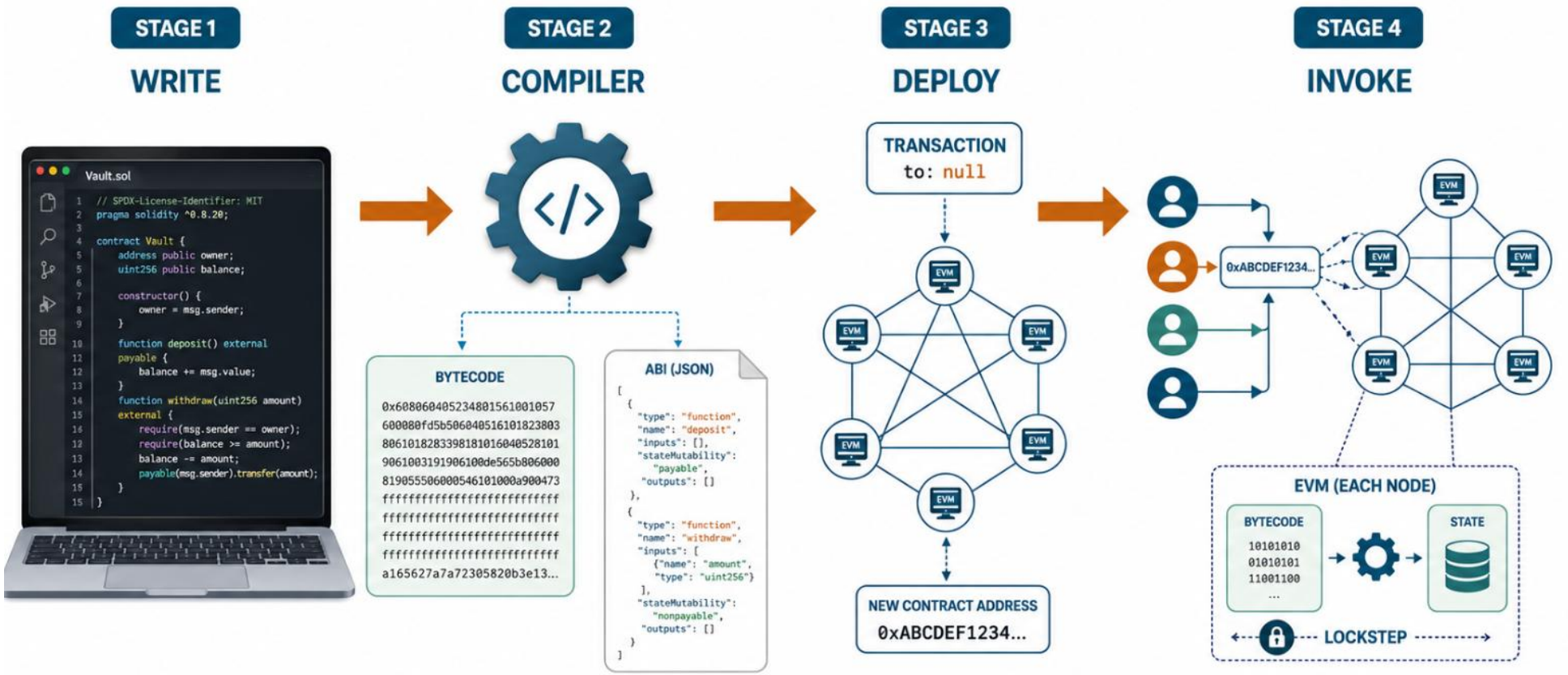
Si los nodos de la red ya están de acuerdo en el estado del ledger (los saldos), y aceptan transacciones que lo modifican... ¿por qué no permitir que esas transacciones ejecuten código arbitrario?

Lo que añade Ethereum a Bitcoin

1. Cada nodo lleva una pequeña máquina virtual: la **EVM (Ethereum Virtual Machine).**
2. Los datos de los contratos se guardan en los bloques igual que los saldos.
3. Cuando alguien invoca un contrato, **TODOS los nodos ejecutan el mismo código con los mismos datos** y deben llegar al mismo resultado.

Resultado:
Ethereum es de facto un ***computador mundial***, una máquina de estados descentralizada, pública y confiable.

La EVM y el ciclo de vida de un contrato



1 WRITE
Developers write smart contract source code in Solidity.

2 COMPILE
The compiler converts Solidity code into bytecode and generates the ABI.

3 DEPLOY
A transaction with no recipient (to: null) creates a new contract on the network at a unique address.

4 INVOKE
Users send transactions to the contract. Every node executes the same bytecode in lockstep within the EVM.

FUNDAMENTAL

Read vs Write — la distinción clave

📖 READ (view / pure)

✓ NO modifica el estado del contrato
✓ NO consume gas
✓ NO requiere transacción ni firma
✓ Respuesta inmediata, sin esperar bloque

Ejemplos:

- `balanceOf(address)`
- `totalSupply()`
- `name(), symbol(), decimals()`
- `getOwner()`

En Etherscan:
pestaña 'Read Contract'

✏️ WRITE (función normal)

✗ SÍ modifica el estado
✗ SÍ consume gas (cuesta dinero)
✗ Requiere firma con clave privada
✗ Esperar a que se incluya en un bloque

Ejemplos:

- `transfer(to, amount)`
- `approve(spender, amount)`
- `mint(to, amount)`
- `withdraw()`

En Etherscan:
pestaña 'Write Contract' (con MetaMask)

EJEMPLO

Counter — el "Hola Mundo" de Solidity

El contrato más sencillo posible para entender Solidity

Una variable contadora que se puede incrementar, decrementar o reiniciar. 4 líneas de lógica. Suficiente para entender variables de estado, funciones de lectura/escritura, y require.

Lo que vamos a ver al ejecutarlo

1. Una variable de estado `count` que vive en la blockchain — no en tu ordenador.

2. Diferencia entre lectura (gratis) y escritura (paga gas, pide firma).

3. Estado global compartido: si alguien hace `increment()`, todos los nodos del mundo verán `count = N+1`.

4. Cómo se valida y propaga ese cambio en la red — el corazón de blockchain.

CÓDIGO

Counter.sol — comentado línea a línea

```
// SPDX-License-Identifier: MIT          ← Cabecera obligatoria con la licencia
pragma solidity ^0.8.20;                 ← Versión del compilador (igual o superior a 0.8.20)

contract Counter {                       ← Declara el contrato (como una clase)

    uint public count;                   ← Variable de estado pública. Vive en la blockchain.
                                         Solidity genera automáticamente un getter "count()".

    function increment() external {      ← Función pública. Modifica estado → cuesta gas.
        count += 1;                      ← Suma 1 al contador. Esto se grabará en blockchain.
    }

    function decrement() external {
        require(count > 0, "No puede ser negativo");   ← Si la condición falla, revierte la tx.
        count -= 1;
    }

    function reset() external {
        count = 0;
    }
}
```

EXPERIMENTO

Counter en Remix — qué pasa al pulsar cada botón

📖 count (botón AZUL)

Es el getter automático de la variable.

✓ Lee directamente del estado del nodo
✓ NO consume gas (no es transacción)
✓ No pide firma
✓ Respuesta inmediata

¿Por qué?
Porque no necesitas que la red llegue a consenso para leer un valor que ya está en el estado actual de los nodos.

✏️ increment / decrement / reset (NARANJA)

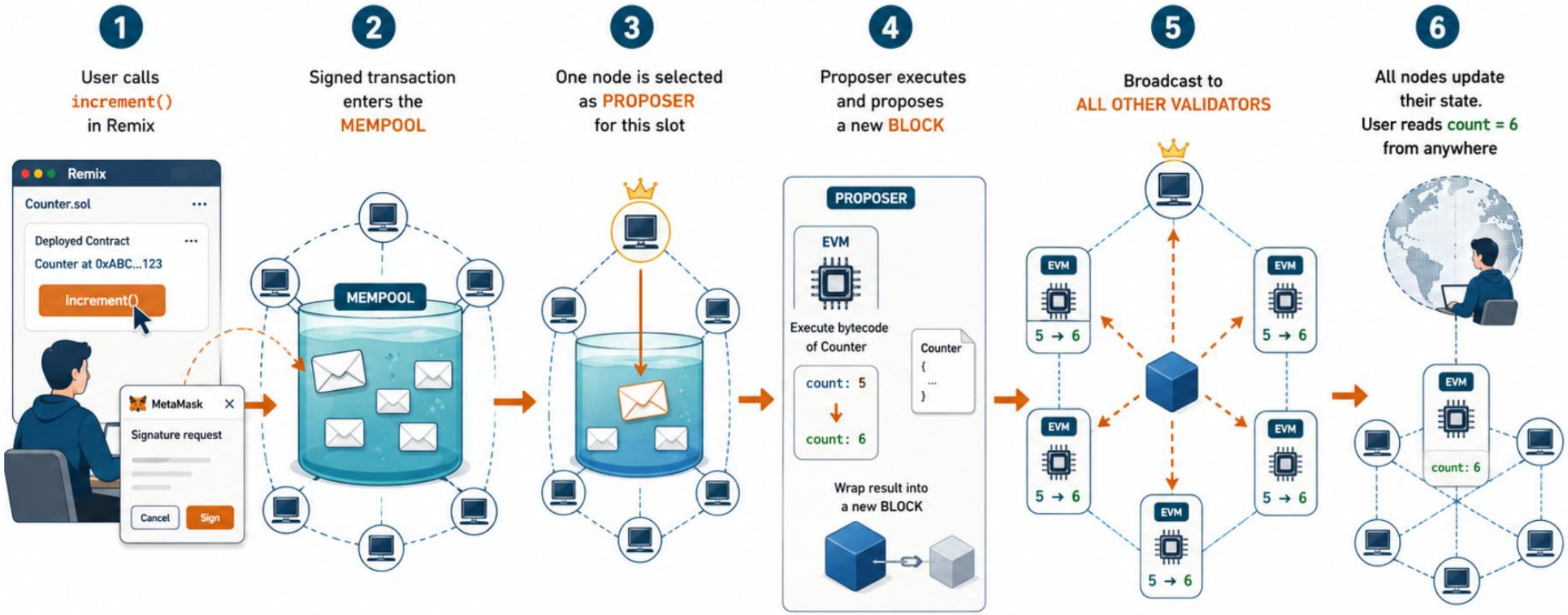
Modifican la variable count.

✗ Cambian el estado del contrato
✗ Consumen gas (~30.000 unidades)
✗ Pide firmar
✗ Hay que esperar a que se incluya en un bloque

¿Por qué?
Porque es un cambio que tiene que aceptar TODA la red, ejecutarse en cada nodo, y propagarse globalmente. Eso es lo que veremos en la siguiente slide.

CICLO COMPLETO

Qué pasa cuando llamas a increment()



1. SIGN
User calls `increment()` in Remix. MetaMask asks for a signature. User signs the transaction.

2. MEMPOOL
The signed transaction is sent to the network and enters the mempool, seen by all nodes.

3. PROPOSER PICKS
For this slot, one node is chosen as proposer. It picks the transaction from the mempool.

4. EXECUTE & PROPOSE BLOCK
The proposer runs the contract bytecode in its EVM. `count` changes from 5 to 6. It packages the state transition into a new block, linking to the previous one.

5. PROPAGATE & VALIDATE
The block is broadcast to all other validators. Each runs the SAME bytecode in their EVM and gets the SAME result: 5 → 6.

6. NEW STATE EVERYWHERE
All nodes update their world state to `count = 6`. Users can read the new value from anywhere.

HERRAMIENTA

¿Qué es Remix?

Un IDE de Solidity que vive en el navegador

Remix (https://remix.ethereum.org) es un entorno de desarrollo web — sin instalar nada — donde puedes escribir, compilar, desplegar y probar smart contracts en cuestión de minutos. Es la herramienta más usada para aprender Solidity.

Lo que ofrece Remix

📝 **Editor Solidity** con resaltado de sintaxis y autocompletado.

⚙️ **Compilador integrado**
— convierte tu código a bytecode + ABI con un clic.

🚀 **Deploy & Run**
— despliega contratos en una red local (JavaScript VM) o en una real (Sepolia, Mainnet) vía MetaMask.

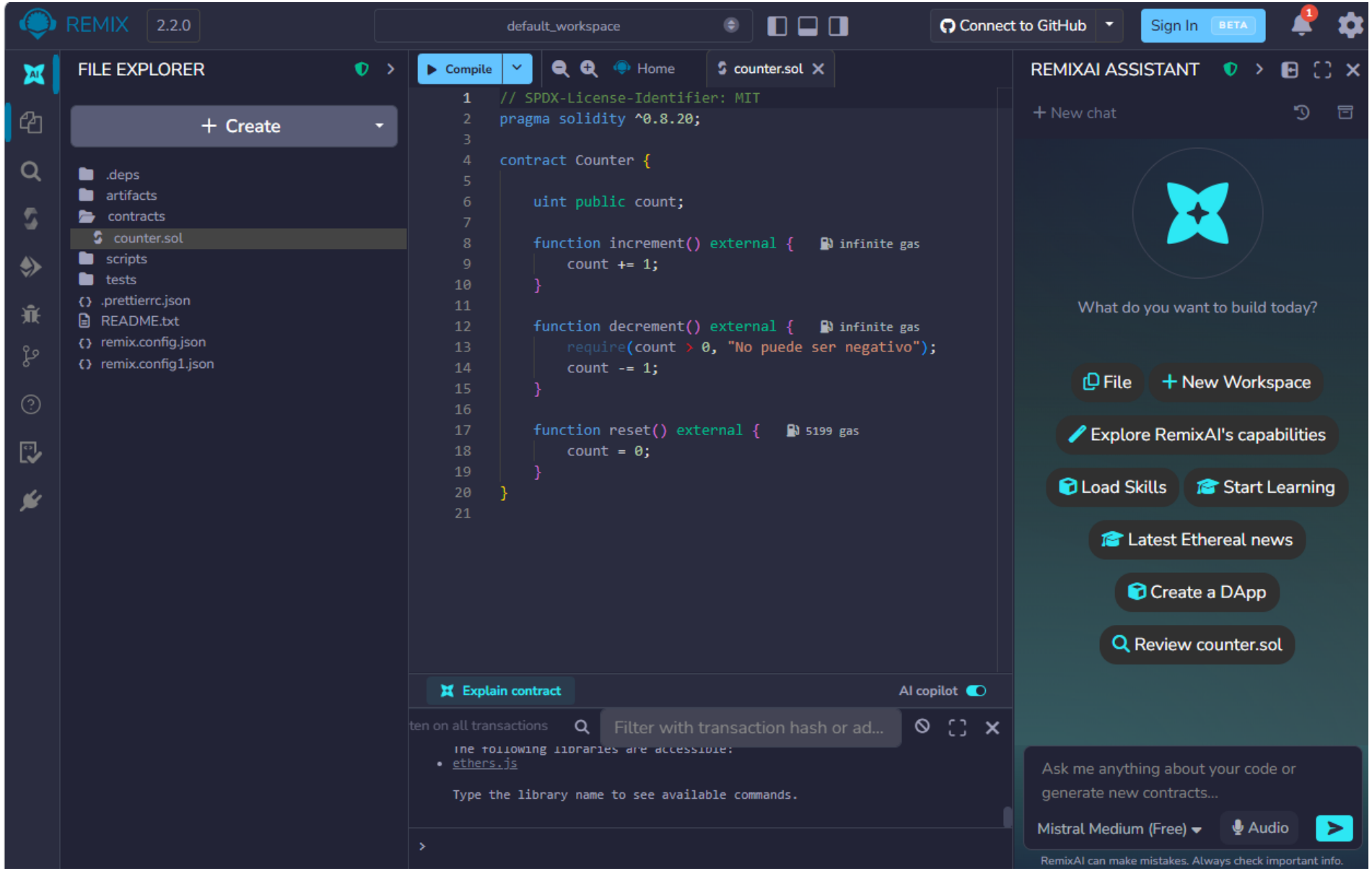
🧪 **Interacción**
— botones para llamar a cada función del contrato sin escribir código JavaScript.

🐛 **Debugger**
— paso a paso de la ejecución, opcodes, cambios de estado, costes de gas.

Gratis y sin instalación. Todo el código se guarda en tu navegador.

HERRAMIENTA

¿Qué es Remix?



PASO A PASO

Desplegar Counter en Remix + Sepolia

1. Ir a `https://remix.ethereum.org`. Borrar los contratos de ejemplo.

2. En el explorador de la izquierda, crear nuevo fichero `Counter.sol` y pegar el código.

3. Abrir el panel **Solidity Compiler** (icono Ⓢ a la izquierda). Verificar que la versión es ≥ 0.8.20. Pulsar **Compile Counter.sol**. Aparece un check verde si todo va bien.

4. Abrir el panel **Deploy & Run Transactions** (icono ◧). En **Environment** seleccionar Injected Provider — MetaMask. Confirmar conexión en el popup.

5. Comprobar que MetaMask está en la red **Sepolia**. Si no, cambiar de red ahora.

6. Pulsar el botón naranja **Deploy**. MetaMask abre popup pidiendo firmar la transacción de despliegue. Confirmar.

7. Esperar ~12 segundos. El contrato aparece en **Deployed Contracts** con su dirección. **¡Ya está desplegado!**

8. Desplegar el contrato: pulsar `count` (azul) → devuelve 0. Pulsar `increment` (naranja) → MetaMask pide firma → esperar bloque → pulsar `count` → ahora devuelve 1.

COSTES

Gas en Smart Contracts — cada operación cuesta

Costes orientativos por operación

Operación	Gas aprox.	Comentario
`ADD, SUB, MUL`	**3 - 5**	Operaciones aritméticas: baratísimas
`SLOAD (leer storage)`	**2.100**	Leer una variable de almacenamiento
`SSTORE (escribir nuevo)`	**20.000**	Escribir por primera vez en un slot — caro
`SSTORE (modificar)`	**5.000**	Modificar valor existente
`LOG (emitir evento)`	**375 + ~750/byte**	Coste medio, escala con datos emitidos
`Despliegue de contrato`	**200.000+**	Depende del tamaño del bytecode

Lección práctica: *los datos en storage son MUY caros. Por eso los contratos serios minimizan al máximo lo que escriben en blockchain y guardan lo demás en eventos (logs) o fuera de cadena.*

EVENTOS

Cómo un Smart Contract "habla" con el mundo exterior

El problema

Un smart contract no puede llamar a un servidor ni enviar un email. Pero las aplicaciones cliente (wallets, frontends, bots) necesitan saber qué pasó: ¿se transfirió un token? ¿se acuñó un NFT? ¿se ejecutó una venta?

La solución: events / logs

El contrato emite eventos que se graban en los logs del bloque. Las aplicaciones se suscriben a esos logs y reaccionan en tiempo real.

```
// Declaración del evento
event Transfer(address indexed from, address indexed to, uint256 value);

// Dentro de la función transfer
emit Transfer(msg.sender, recipient, amount);
```

Recuerda: *los logs los viste en la pestaña 'Logs' de Etherscan en una transacción ERC-20. Ahora ya sabes de dónde salen.*

Lo que hicieron posible los Smart Contracts

Estándares de tokens

ERC-20 *(2015)* — Tokens fungibles. Cualquier proyecto puede crear su propia 'moneda' con 100 líneas de código. USDC, DAI, UNI, LINK... todo el ecosistema DeFi depende de este estándar.

ERC-721 *(2018)* — Tokens no fungibles (NFTs). Cada token es único e identificable. CryptoKitties, Bored Apes, arte digital, identidad on-chain.

ERC-1155 *(2019)* — Híbrido. Un mismo contrato puede tener miles de tipos de tokens, fungibles y no fungibles. Diseñado para juegos: cada espada es única (NFT) pero hay 1.000 pociones idénticas (fungible).

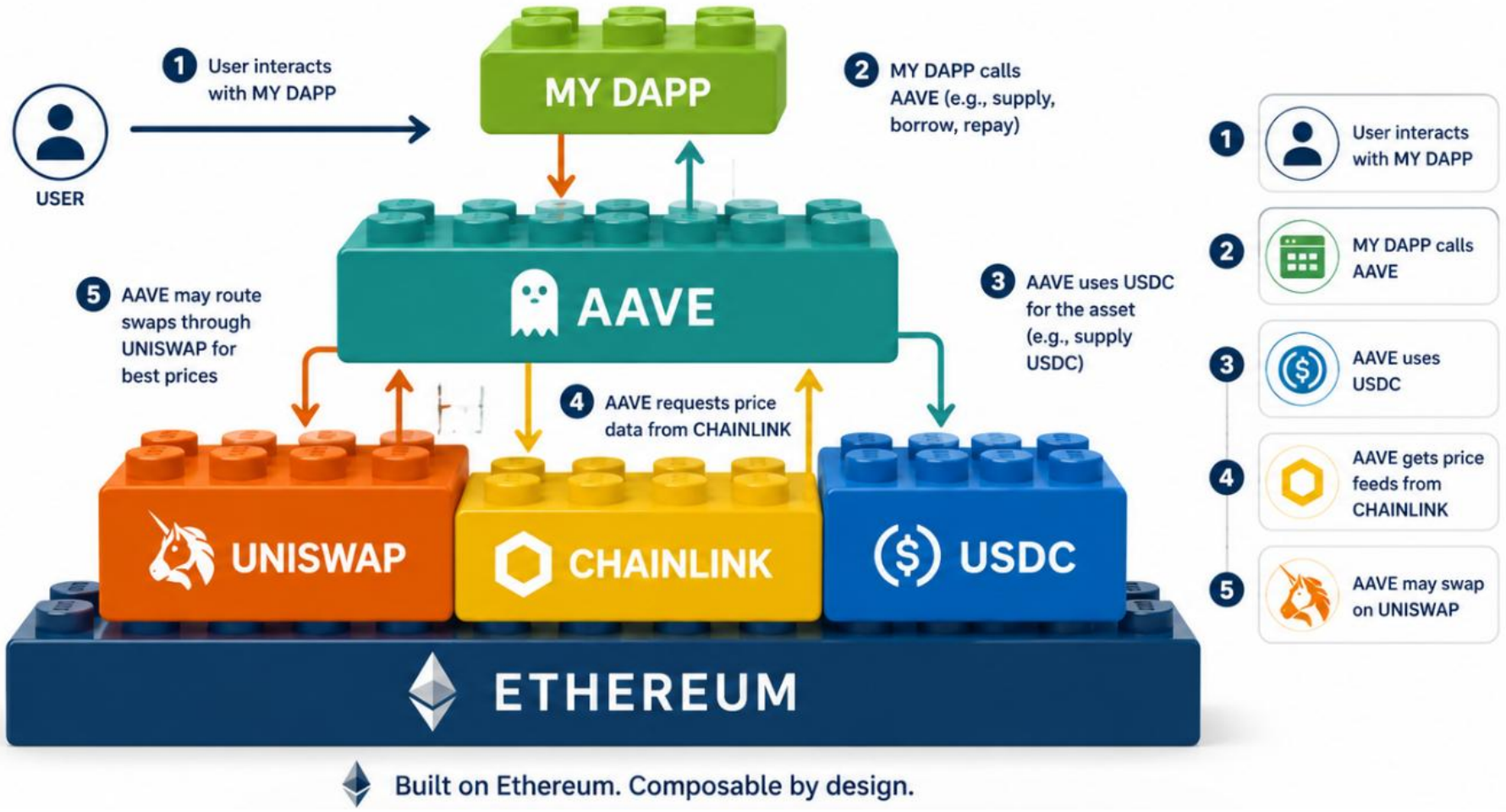
Y a partir de ahí:
DeFi (Uniswap, Aave, Compound), DAOs (MakerDAO, Uniswap DAO), bridges, DEX, ICOs/IDOs, RWA tokenizado, identidad descentralizada, GameFi...

SUPERPODER

Composabilidad — "Lego Money"

Por qué los smart contracts cambiaron las finanzas

Cada smart contract expone funciones públicas. Otro contrato puede llamarlas como una API. Eso permite combinar contratos como piezas de Lego para crear productos financieros nuevos. Sin permiso. Sin pedir nada a nadie.



MATIZ

Inmutabilidad… con asterisco

Lo que se suele decir

"Un smart contract es inmutable: una vez desplegado, ni el autor puede cambiar el código."

La realidad

El bytecode SÍ es inmutable. Pero el 90% de los contratos serios en producción usan patrones que permiten actualizar la lógica:

Proxy pattern (delegatecall): el contrato "de cara al usuario" delega la ejecución a otro contrato (la implementación). Si quieres actualizar la lógica, despliegas una nueva implementación y cambias el puntero. Lo usan Uniswap, Aave, OpenSea...

Diamond pattern (EIP-2535): varios contratos de implementación ("facets") detrás de un único proxy. Permite añadir/quitar funciones sin redesplegar todo.

Mecanismos de gobernanza: los cambios pueden requerir aprobación de una multisig, un timelock o una votación de la DAO.

Moraleja: *"inmutable" es más una elección de diseño que una propiedad automática. Antes de usar un contrato, comprueba si es upgradeable y quién controla la actualización.*

SEGURIDAD

Cuando un bug cuesta cientos de millones

Tres hacks que marcaron la historia

The DAO
2016

60M USD

Una DAO con 150M USD en ETH fue vaciada por una vulnerabilidad de reentrancy: el atacante llamaba a withdraw() recursivamente antes de que se actualizara su saldo. Forzó el primer hard-fork de Ethereum (ETH vs ETC).

Lección: *Las llamadas externas son peligrosas. Patrón Checks-Effects-Interactions.*

Parity Multisig
2017

280M USD

Un usuario llamó a una función de inicialización en una librería compartida y se hizo dueño de ella. Luego invocó selfdestruct y borró la librería que usaban miles de wallets multisig. 280M USD bloqueados PARA SIEMPRE.

Lección: *Inicializadores y librerías compartidas son una bomba. Cuidado con selfdestruct.*

Wormhole
2022

320M USD

El bridge entre Ethereum y Solana tenía un bug en la verificación de firmas de los guardianes. El atacante consiguió mintear 120.000 ETH en Solana sin haber bloqueado los reales en Ethereum.

Lección: *Verificar TODAS las firmas correctamente. Los bridges son superficies de ataque enormes.*

FUNDAMENTAL

Control de acceso — quién puede ejecutar qué

El problema

Si despliegas un contrato con una función pública withdraw() y no proteges quién la puede llamar, cualquiera puede vaciarlo. Tu contrato debe distinguir entre 'usuarios cualesquiera' y 'el dueño'.

Patrón Ownable

```
address public owner;

constructor() { owner = msg.sender; }  // quien despliega es el owner

modifier onlyOwner() {
    require(msg.sender == owner, "No autorizado");
    _;
}

function withdraw() public onlyOwner {  // solo el owner puede llamar
    payable(owner).transfer(address(this).balance);
}
```

En la práctica: *se usa el contrato `Ownable` de OpenZeppelin (estándar de seguridad), que añade transferOwnership y eventos.*

?

Pregunta abierta para reflexionar

¿Por qué no hay números aleatorios en los Smart Contracts?

Pensadlo en parejas, 5 minutos. Discusión en la siguiente slide.

RESPUESTA

Determinismo: por qué no hay aleatoriedad

El motivo de fondo

Todos los nodos de la red ejecutan el MISMO código con los MISMOS datos. Si una operación devolviera un número aleatorio, cada nodo obtendría un resultado distinto y la red no llegaría a consenso. La blockchain dejaría de funcionar.

Cómo se simula la aleatoriedad

1. Hash de bloques futuros — usar block.prevrandao. Inseguro: el validador puede manipularlo.

2. Commit-Reveal
— los participantes envían primero un hash (commit) y después el valor real (reveal). Más seguro pero requiere dos transacciones.

3. Chainlink VRF (Verifiable Random Function)
— el estándar de la industria. Un oráculo descentralizado genera un número aleatorio criptográficamente verificable. El contrato puede comprobar que es genuino. Lo usan miles de protocolos para juegos, sorteos, mints aleatorios de NFTs...

Anécdotas que marcaron el ecosistema

FIT VITALIK (2017)

Una parodia de ICO lanzada para "recaudar fondos para que Vitalik se ponga en forma". Llegó a cotizar en exchanges reales. El smart contract se desplegó en:

0xe250d7fd146e009dcc2ff367cad01b2ac3c70266

Demuestra hasta dónde llegó la fiebre de las ICOs en 2017: la gente compraba CUALQUIER token con tal de no quedarse fuera.

THE DAO (2016)

La primera DAO de la historia. Recaudó 150M USD (12% del ETH circulante en ese momento). Un atacante explotó un bug de reentrancy y vació 60M USD.

La comunidad se dividió: ¿revertimos el hack o aceptamos "code is law"? La votación dio lugar al primer hard-fork: Ethereum (ETH, con el rescate) y Ethereum Classic (ETC, sin él).

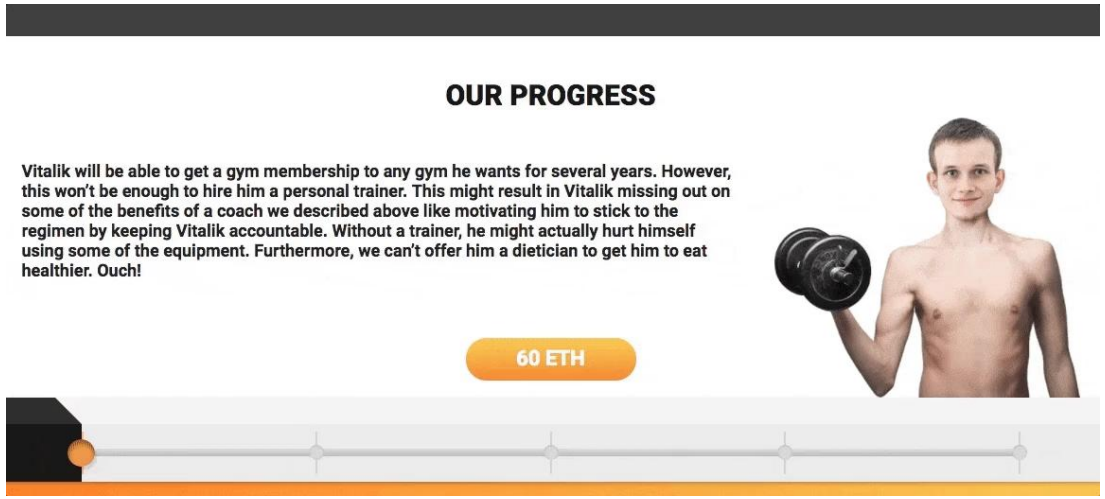
Fue la pérdida de la inocencia del ecosistema: los smart contracts no son magia, son código con bugs.

Cybercultura - Fit Vitalik

la campaña llamada FitVitalik fue una **parodia de ICO** lanzada en noviembre de 2017 con el fin de recaudar fondos para que Vitalik Buterin “se ponga en forma”

Llegó a cotizar en exchanges, el Smart contract estaba en esta address:

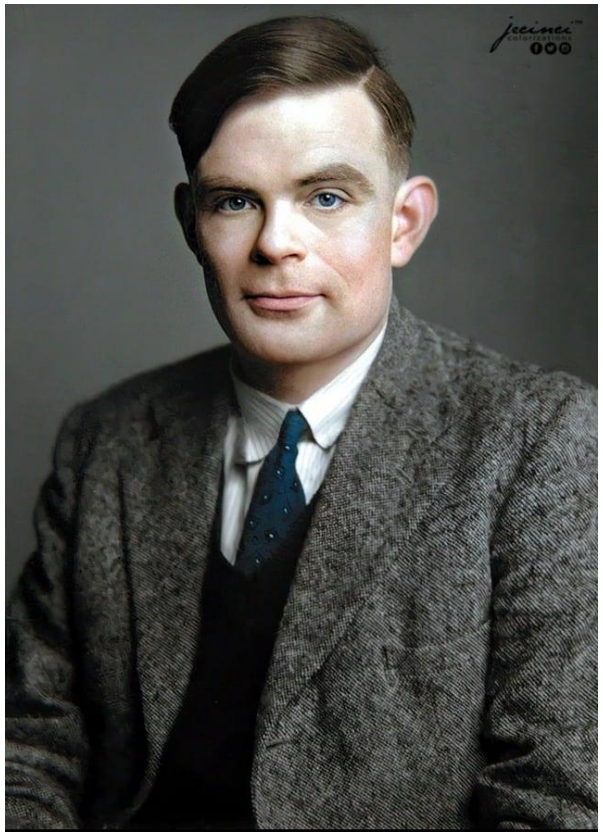
0xe250d7fd146e009dcc2ff367cad01b2ac3c70266

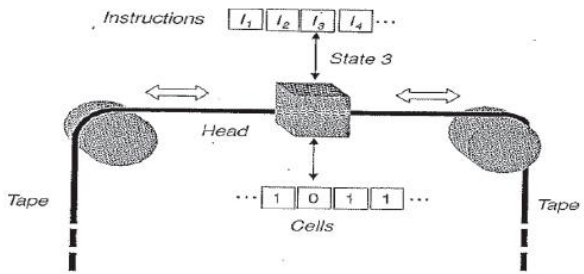


Cultura General

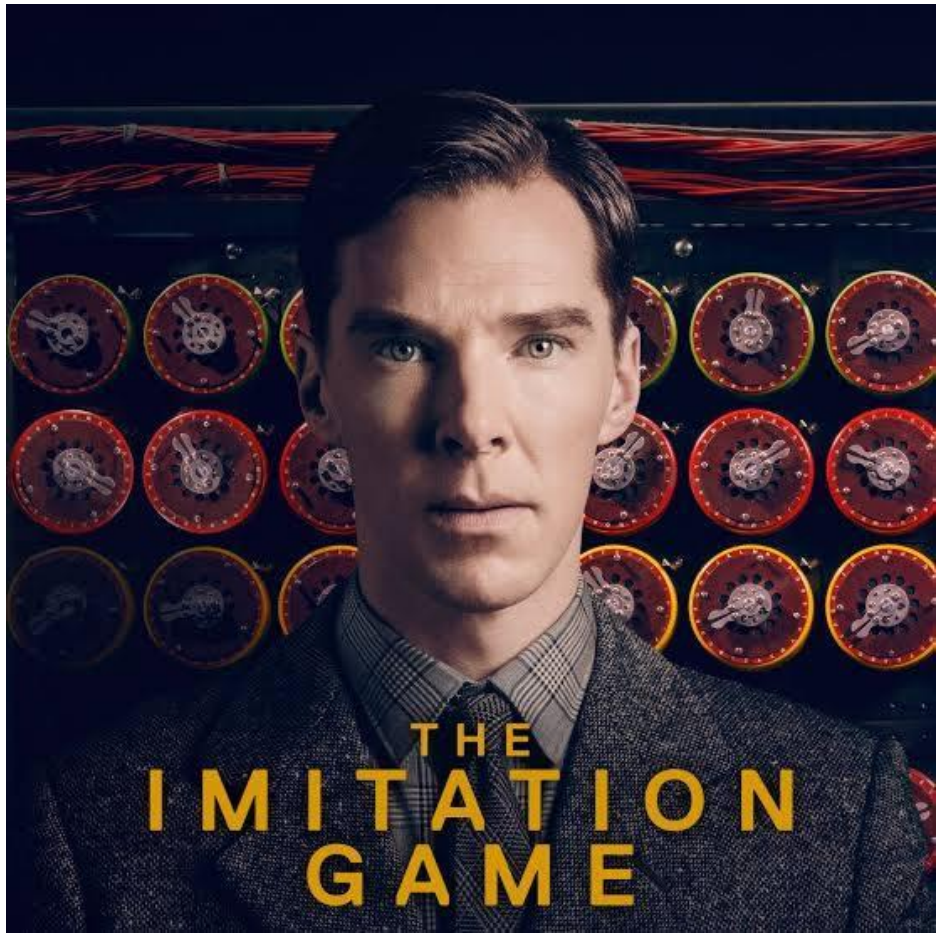
Alan Turing (1912 – 1954)

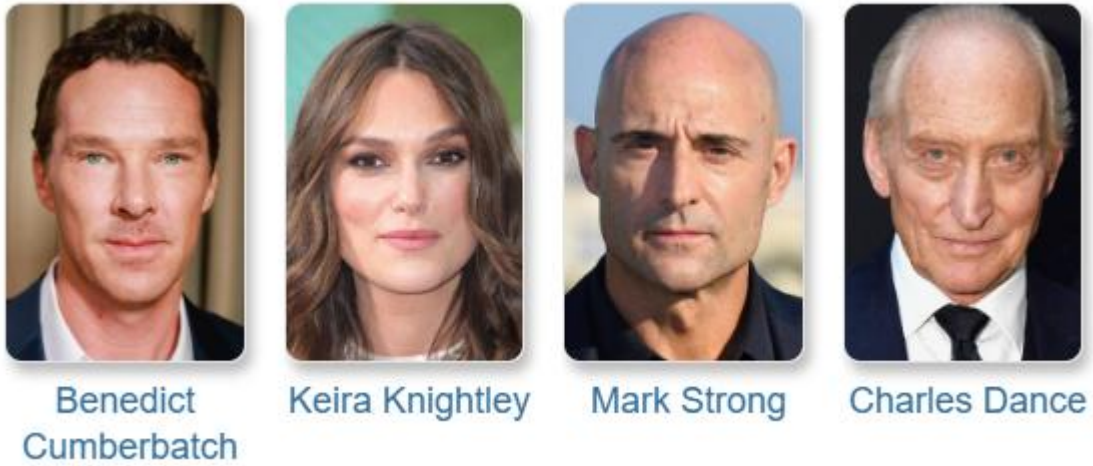
- **Alan Turing** Matemático, lógico y criptógrafo británico.
- Propuso en 1936 la **máquina de Turing**, base teórica de los computadores modernos.
- Durante la Segunda Guerra Mundial dirigió el equipo que **rompió el código Enigma**, acortando la guerra en años.
- En 1950 planteó el **Test de Turing**, primer intento formal de medir inteligencia artificial.
- Fue perseguido por su homosexualidad y murió en 1954.
- Hoy es considerado el **padre de la computación y la IA**.





Descifrando Enigma





Biopic sobre el matemático británico Alan Turing, famoso por haber descifrado los códigos secretos nazis contenidos en la máquina Enigma, lo cual determinó el devenir de la II Guerra Mundial (1939-1945) en favor de los Aliados.

Lejos de ser admirado como un héroe, Turing fue acusado y juzgado por su condición de homosexual en 1952.

CÓDIGO

Vault.sol — qué vamos a desplegar

Caja fuerte temporal: bloquea ETH durante un tiempo

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract Vault {
    mapping(address => uint) public deposits;       // saldo bloqueado por usuario
    mapping(address => uint) public unlockTime;     // timestamp de desbloqueo

    event Deposited(address indexed user, uint amount, uint unlockAt);
    event Withdrawn(address indexed user, uint amount);

    function deposit(uint lockSeconds) external payable {
        require(msg.value > 0, "Envia ETH");
        deposits[msg.sender] += msg.value;
        unlockTime[msg.sender] = block.timestamp + lockSeconds;
        emit Deposited(msg.sender, msg.value, unlockTime[msg.sender]);
    }

    function withdraw() external {
        require(block.timestamp >= unlockTime[msg.sender], "Aun bloqueado");
        uint amount = deposits[msg.sender];
        deposits[msg.sender] = 0;                   // efecto ANTES de la interacción
        payable(msg.sender).transfer(amount);
        emit Withdrawn(msg.sender, amount);
    }
}
```

Repasar el patrón Checks-Effects-Interactions (línea naranja): primero comprobar, luego modificar estado, luego interactuar. Es lo que evita reentrancy.

PRÁCTICA 1

Leer un Smart Contract real en Etherscan

Antes de escribir el nuestro, vamos a ver uno de verdad

1. Ir a sepolia.etherscan.io y buscar el contrato del USDC de Sepolia:
`0x1c7D4B196Cb0C7B01d743Fbc6116a902379C7238`

2. Ver la pestaña **Contract**. Identificar: ¿está verificado? ¿qué versión de Solidity usa? ¿Cuántas funciones tiene?

3. En **Read Contract**: invocar `balanceOf` con tu propia dirección. ¿Cuántos USDC tienes? ¿Es gratis hacer esta llamada?

4. Buscar la función `transfer` en **Write Contract**. ¿Qué parámetros pide? ¿Por qué esta sí pediría firmar con MetaMask?

5. Mirar los **Events** que emite el contrato. Ver una transacción reciente y localizar el evento `Transfer` con sus tres parámetros (from, to, value).

PRÁCTICA 2

Desplegar nuestro propio Vault

Pasos en Remix + Sepolia

1. Acceder a Remix: `https://remix.ethereum.org/`
2. Borrar los contratos de ejemplo. Crear nuevo fichero `Vault.sol` y pegar el código.
3. Compilar (icono Solidity Compiler). Verificar que no hay errores. Ver el bytecode generado.
4. En la pestaña Deploy: **Environment → Injected Provider (MetaMask)**. Comprobar que estás en Sepolia.
5. Pulsar Deploy. MetaMask te pide firmar la transacción. Confirmar y esperar a que se incluya en un bloque.
6. Probar `deposit(60)` con 0.001 ETH. Intentar `withdraw()` inmediatamente — debe fallar ("Aun bloqueado").
7. Esperar 60 segundos. Volver a intentar `withdraw()` — ahora funciona. Verificar el balance recuperado.
8. Buscar la dirección del contrato en sepolia.etherscan.io y ver tus transacciones + eventos emitidos.